

Taller NoSQL

Integrantes:

- Alexis Romero - CI 4.653.441-7
- Andrés Fernández - CI 4.859.574-0
- Dámaso Tor - CI 4.508.724-7
- Mateo Sparano - CI 5.545.828-2

1. Realidad en la cual utilizaremos MongoDB

1.1 Descripción general

Proponemos utilizar un sistema de base de datos no relacional con MongoDB para implementar un e-commerce de una tienda que vende diversos artículos.

Creemos que una base de datos no relacional tiene algunas ventajas para un proyecto así.

Flexibilidad del esquema

La tienda venderá artículos diferentes. Y debe mostrar información a los visitantes con datos que son diferentes entre las distintas categorías de productos.

En ese sentido, en la descripción de una camiseta, tendremos los datos de talla, color y marca. En cambio, en otros tipos de artículos como muebles, tendremos características como altura, largo, profundidad, material.

Y a futuro, la tienda podría expandir su catálogo a otros rubros, sin necesidad de cambiar el esquema de la base de datos.

Por tanto, la flexibilidad del esquema que tiene MongoDB permitiría guardar para cada categoría de artículos los atributos que necesita realmente, sin la complicación y los grandes JOIN que serían necesarios en una base de datos relacional para hacerlo.

Escalado horizontal

Los e-commerce experimentan cargas de trabajo impredecibles, con picos y valles de acuerdo a la afluencia de los compradores. El soporte nativo de MongoDB para el escalado horizontal permite afrontar este reto con soluciones en la nube que permiten agregar y

reducir servidores de base de datos de acuerdo a la demanda, sin el costo de los recursos que se necesitan en los picos durante todo el tiempo.

Las bases de datos que solo soportan escalado vertical requiere contratar servidores de alto desempeño durante toda su operación.

Este escalado horizontal permite también fortalecer la alta disponibilidad de la base de datos, evitando un punto único de falla. De esta manera, evitamos fallas o saturaciones en picos extremos de demanda, que causan downtime.

Las bases NoSQL priorizan la disponibilidad sobre la consistencia estricta, lo que es ideal para lecturas rápidas en e-commerce, donde un pequeño retraso puede perder ventas.

Estructura de datos

La estructura de datos documental de MongoDB se alinea mejor con el modelado de objetos en nuestro código. Esto nos permite representar entidades complejas, como un cliente con múltiples direcciones y pedidos, en un solo documento, simplificando las consultas y mejorando el rendimiento al evitar JOINS complejos.

Tipo de consultas

MongoDB además de ofrecer operaciones atómicas para agregar, eliminar o actualizar datos en una colección, también ofrece el Aggregation Pipeline, que nos permitirá realizar búsquedas avanzadas de productos. Esto nos habilita a ofrecer al visitante búsquedas avanzadas de acuerdo a criterios complejos.

1.2 Casos de uso

1.2.1 Filtro avanzado de artículos

Actor: Visitante

Descripción: El visitante desea filtrar los artículos por distintas características o especificaciones de los productos. Asimismo, se podrá filtrar por sucursales en las que haya stock del artículo.

Ventajas de utilizar MongoDB:

La estructura de datos documental permite concentrar en un documento los datos que se requieren del producto, incluyendo arreglos con sucursales donde hay determinado stock. Esto nos evita unir tablas, que es una operación costosa en grandes cantidades de datos. Las consultas de MongoDB permiten condiciones complejas sobre campos anidados y elementos dentro de arreglos de forma nativa.

ID CU:	CU-001
Nombre CU:	Filtro avanzado de artículos
Actor principal:	Visitante
Objetivo:	Permitir al Visitante aplicar filtros a la lista de artículos visibles, basándose en características, especificaciones y disponibilidad de stock en sucursales.
Precondiciones:	El Visitante está navegando en la sección artículos/productos del sitio. Existen artículos cargados en el sistema.
Postcondiciones:	El Sistema muestra una lista de artículos actualizada y acotada a los artículos que cumplen con todos los criterios de filtro seleccionados por el Visitante.
Flujo principal:	1. El Visitante ingresa a la vista de artículos y ve la lista completa. 2. El Visitante selecciona uno o varios filtros de características o especificaciones (ej. "Color: Rojo", "Volumen: 750ml"). 3. El Visitante selecciona una o varias Sucursales para filtrar por stock. 4. El Visitante presiona el botón "Aplicar Filtros". 5. El Sistema ejecuta una única consulta a la base de datos que aplica las condiciones sobre las características y busca coincidencias dentro del arreglo de inventario por sucursal. Recibe el conjunto acotado de artículos. 6. El Sistema actualiza y muestra la lista filtrada de artículos al Visitante.
Flujos alternativos:	A.1. Restablecer filtros. (Antes del paso 4 o después del paso 6.) 1. El Visitante presiona el botón "Limpiar filtros". 2. El Sistema elimina todas las selecciones de filtros de Características y Sucursales. 3. El Sistema ejecuta una consulta para obtener el listado completo original de artículos. 4. El Sistema actualiza la vista mostrando el catálogo completo.
Flujos de excepción:	E.1. No se encuentran resultados. (Durante el paso 5). 1. Se muestra el mensaje: "No se encontraron artículos que coincidan con los filtros seleccionados".
Requisitos no funcionales:	1. La consulta de filtrado debe devolver resultados en menos de 2 segundos. (Requisito de rendimiento).

1.2.2 Carrito de compras persistente y autolimpiable

Actor: Visitante

Descripción: Un usuario navega por el sitio y agrega productos a su carrito. El carrito debe recordar estos productos entre visitas (sesiones), manejar la adición y eliminación de artículos. Si el visitante no concreta la compra en una determinada cantidad de días, el carrito se autolimpia.

Ventajas de utilizar MongoDB:

Todo el carrito del usuario puede ser un único documento. Agregar, quitar o actualizar la cantidad de un producto es una sola operación atómica sobre ese documento. Esto es extremadamente rápido y reduce la carga en la base de datos, lo cual es crucial durante los picos de tráfico.

Asimismo, MongoDB tiene una funcionalidad nativa llamada índices TTL (Time-To-Live), que permite configurar un índice en un campo de fecha para que MongoDB elimine automáticamente los documentos cuando haya pasado un cierto tiempo.

ID CU:	CU-002
Nombre CU:	Gestionar carrito persistente
Actor principal:	Visitante
Objetivo:	Permitir al Visitante mantener un registro de los productos deseados a través de múltiples sesiones. El Sistema limpia automáticamente el carrito si la compra no se completa en determinada cantidad de días.
Precondiciones:	El Visitante tiene una cookie de sesión (o está logueado) que permite identificar su carrito. El catálogo de productos está disponible.
Postcondiciones:	El carrito del Visitante está actualizado con las modificaciones realizadas (productos/cantidades), y el campo de fecha de última actividad está actualizado.
Flujo principal:	1. El Visitante navega por el sitio. 2. El Visitante pulsa "Añadir al Carrito" en un producto. 3. El Sistema localiza o crea al documento del carrito del Visitante en la base de datos. 4. El sistema realiza una única operación atómica para: a. Añadir el producto (si es nuevo) o incrementar su cantidad (si ya existe). b. Actualizar el campo <i>última_actividad</i> del documento del carrito a la hora actual. 5. El Sistema muestra un mensaje de confirmación y el subtotal actualizado del carrito.
Flujos alternativos:	A.1. Eliminar artículo (Se produce después del paso 5). 1. El Visitante selecciona "Eliminar" en un artículo dentro del

	carrito. 2. El Sistema realiza una operación atómica para remover el subdocumento del artículo y actualizar el subtotal y la última actividad. 3. El Sistema muestra un mensaje de confirmación y el subtotal actualizado del carrito.
Flujos de excepción:	E.1. Producto agotado (Durante el paso 4). 1. Si el producto no tiene más stock, la operación atómica falla. 2. El Sistema muestra un mensaje de error y el carrito no se modifica.
Requisitos no funcionales:	1. Persistencia de sesión. Los artículos del carrito deben persistir por un período definido incluso si el Visitante cierra su navegador o dispositivo. 2. Autolimpieza. Si un carrito no registra actividad (no hay adiciones o eliminaciones) durante 30 días (ejemplo), debe ser eliminado automáticamente del sistema.

2. Diseño de la base de datos en MongoDB

2.1 Colecciones:

2.1.1 Artículos

Descripción

Esta colección almacena la información de cada artículo en venta. Cada documento representa un producto único, con campos flexibles para adaptarse a diferentes categorías (ropa, muebles, electrónicos). Esto permite esquemas variables sin alterar la estructura global.

Estructura:

Si bien el formato BSON no permite comentarios, los agregamos para explicar cada campo.

```
db.createCollection("articulos", {
  validator: {
    $jsonSchema: {
```

```

bsonType: "object",
required: ["nombre", "precio", "categoria", "creado_en", "actualizado_en"],
properties: {
  nombre: {
    bsonType: "string",
    minLength: 1,
    description: "Debe ser un string no vacío"
  },
  descripcion: {
    bsonType: "string",
    description: "Debe ser un string si se incluye"
  },
  precio: {
    bsonType: "number",
    minimum: 0,
    description: "Debe ser un número no negativo"
  },
  categoria: {
    bsonType: "string",
    minLength: 1,
    description: "Debe ser un string no vacío"
  },
  atributos: {
    bsonType: "object",
    description: "Debe ser un objeto flexible sin restricciones internas"
  },
  imagenes: {
    bsonType: "array",
    items: {
      bsonType: "string",
      pattern: "^https?://",
      description: "Cada elemento debe ser una URL válida"
    }
  },
  inventario: {
    bsonType: "array",
    items: {
      bsonType: "object",
      required: ["sucursal_id", "stock", "ultima_actualizacion"],
      properties: {
        sucursal_id: { bsonType: "objectId" },
        stock: { bsonType: "number", minimum: 0 }, // Stock no negativo
        ultima_actualizacion: { bsonType: "date" }
      }
    }
  },
  creado_en: {
    bsonType: "date",
    description: "Debe ser una fecha válida"
  },
  actualizado_en: {
    bsonType: "date",
    description: "Debe ser una fecha válida"
  }
}
},
validationLevel: "strict",
validationAction: "error",

```

```

indexes: [ {
  "inventario.sucursal_id": 1,
  "inventario.stock": 1
}, {
  name: "idx_inventario_sucursal_stock"
}]
});

```

Explicación

Los campos como "atributos" son objetos dinámicos, permitiendo agregar propiedades nuevas sin migraciones. El arreglo "inventario" embebe el stock por sucursal para consultas rápidas en filtros, evitando joins frecuentes.

Agregamos un índice compuesto entre inventario.sucursal_id e inventario.stock para agilizar consultas del Caso de Uso CU-001, en el que se filtren artículos por su stock en determinada sucursal.

2.1.2 Carritos

Descripción

Colección para manejar carritos de compras persistentes por visitante. Cada documento representa un carrito único, identificado por una cookie de sesión o ID de usuario.

Usamos TTL para autolimpieza basada en inactividad.

Estructura:

```

db.createCollection("carritos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["visitante_id", "registrado", "total", "ultima_actividad",
"creado_en"],
      properties: {
        visitante_id: {
          bsonType: "string",
          minLength: 1,
          description: "Debe ser un ID de visitante no vacío"
        },
        registrado: {
          bsonType: "boolean",
          description: "Si el usuario es registrado, visitante_id=userID, si no,
cookie de sesión"
        },
        items: {
          bsonType: "array",
          items: {
            bsonType: "object",
            required: ["articulo_id", "cantidad", "precio_unitario", "subtotal"],
            properties: {
              articulo_id: { bsonType: "objectId" },
              cantidad: { bsonType: "number", minimum: 1 },

```

```

        precio_unitario: { bsonType: "number", minimum: 0 },
        subtotal: { bsonType: "number", minimum: 0 }
    }
},
total: {
    bsonType: "number",
    minimum: 0,
    description: "Debe ser un número no negativo"
},
ultima_actividad: {
    bsonType: "date",
    description: "Debe ser una fecha válida. Se establecerá una TTL"
},
creado_en: {
    bsonType: "date",
    description: "Debe ser una fecha válida"
}
}
},
validationLevel: "strict",
validationAction: "error",
indexes: [ { key: { ultima_actividad: 1 }, expireAfterSeconds: 2592000 } ]
});

```

Explicación

Todo el carrito se almacena en un solo documento para operaciones atómicas (ej., agregar/quitar items). El campo "ultima_actividad" permite configurar un índice TTL para eliminar carritos inactivos automáticamente después de 30 días, reduciendo el tamaño de la colección con el tiempo.

2.1.3 Sucursales

Descripción

Almacena información de las sucursales físicas, usada para referencias en inventario y filtros por stock.

Estructura:

```

db.createCollection("sucursales", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["nombre", "direccion", "ciudad", "creado_en"],
            properties: {
                nombre: {
                    bsonType: "string",
                    minLength: 1,
                    description: "Debe ser un string no vacío"
                },
                direccion: {
                    bsonType: "string",

```



```

        minLength: 1,
        description: "Debe ser un string no vacío"
    },
    ciudad: {
        bsonType: "string",
        minLength: 1,
        description: "Debe ser un string no vacío"
    },
    horarios: {
        bsonType: "string",
        description: "Debe ser un string si se incluye"
    },
    creado_en: {
        bsonType: "date",
        description: "Debe ser una fecha válida"
    }
}
},
validationLevel: "strict",
validationAction: "error"
});

```

Explicación

Colección simple y estática, ya que las sucursales no cambian frecuentemente. Sirve como referencia para evitar duplicar datos en "inventario" de artículos.

2.1.4 Usuarios

Descripción

Si se implementa login, almacena perfiles de usuarios con referencias a carritos u órdenes.

Estructura:

```

db.createCollection("usuarios", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["email", "nombre", "creado_en"], // Campos obligatorios
            properties: {
                email: {
                    bsonType: "string",
                    pattern: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$",
                    description: "Debe ser un email válido (ej., usuario@dominio.com)"
                },
                rol: {
                    bsonType: "string",
                    description: "Si es usuario, admin, etc..."
                },
                nombre: {
                    bsonType: "string",
                    minLength: 1,

```

```

        description: "Debe ser un string no vacío"
    },
    direcciones: {
        bsonType: "array",
        items: {
            bsonType: "object",
            required: ["calle", "ciudad"],
            properties: {
                calle: { bsonType: "string", minLength: 1 },
                ciudad: { bsonType: "string", minLength: 1 }
            }
        }
    },
    creado_en: {
        bsonType: "date",
        description: "Debe ser una fecha válida"
    }
}
},
validationLevel: "strict",
validationAction: "error"
});

```

Explicación

Embebemos direcciones para consultas rápidas.

2.1.5 Órdenes

Descripción

Esta colección almacena las órdenes de compra completadas por los clientes. Cada documento representa una orden única, capturando el estado del carrito en el momento de la compra para mantener la integridad histórica (ej., precios y cantidades fijas, independientemente de cambios posteriores en los productos). Esto facilita reportes, seguimiento de ventas y análisis de datos, como ventas por sucursal o cliente.

La colección crecerá con el volumen de ventas, pero las operaciones principales son inserts (al finalizar compra) y lecturas para seguimiento.

Estructura:

```

db.createCollection("ordenes", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["usuario_id", "items", "total", "metodo_pago", "estado",
"fecha_pedido", "fecha_actualizacion"],
            properties: {
                usuario_id: {
                    bsonType: ["objectId"],
                    description: "Debe ser ObjectId"
                }
            }
        }
    }
});

```

```

    },
    items: {
      bsonType: "array",
      minItems: 1, // Al menos un ítem para una orden válida
      items: {
        bsonType: "object",
        required: ["articulo_id", "cantidad", "precio_unitario", "subtotal"],
        properties: {
          articulo_id: { bsonType: "objectId" },
          cantidad: { bsonType: "number", minimum: 1 },
          precio_unitario: { bsonType: "number", minimum: 0 },
          subtotal: { bsonType: "number", minimum: 0 }
        }
      }
    },
    total: {
      bsonType: "number",
      minimum: 0
    },
    sucursal_id: {
      bsonType: ["objectId", "null"],
      description: "Debe ser ObjectId o null"
    },
    direccion_envio: {
      bsonType: ["object", "null"],
      required: ["calle", "ciudad", "codigo_postal"],
      properties: {
        calle: { bsonType: "string", minLength: 1 },
        ciudad: { bsonType: "string", minLength: 1 },
        codigo_postal: {
          bsonType: "string",
          description: "Debe ser un código postal numérico"
        }
      }
    },
    metodo_pago: {
      enum: ["tarjeta", "efectivo", "transferencia"],
      description: "Debe ser uno de los métodos permitidos"
    },
    estado: {
      enum: ["pendiente", "procesando", "enviado", "entregado", "cancelado"],
      description: "Debe ser uno de los estados permitidos"
    },
    fecha_pedido: { bsonType: "date" },
    fecha_actualizacion: { bsonType: "date" }
  }
},
validationLevel: "strict",
validationAction: "error",
indexes: [
  { key: { usuario_id: 1 }, name: "idx_usuario_id" },
  { key: { fecha_pedido: -1 }, name: "idx_fecha_pedido", unique: false },
  { key: { estado: 1 }, name: "idx_estado" },
  { key: { sucursal_id: 1 }, name: "idx_sucursal_id" }
]
});

```

Explicación

Los "items" se embeben como arreglo para operaciones atómicas y consultas rápidas (ej., ver detalles de orden sin joins). Esto captura los detalles de la compra, evitando inconsistencias si un artículo cambia precio o stock después.

Referenciamos "sucursal_id" para asociar con retiros en tienda, "usuario_id" para un seguimiento por cliente, y "articulo_id" para enlazar con productos (usando \$lookup en agregaciones si se necesitan detalles completos).

Campos como "estado" permiten actualizaciones atómicas con \$set.

Para optimización, indexar "usuario_id", "fecha_pedido" y "estado" para consultas frecuentes como órdenes pendientes o ventas mensuales.

2.2 Relaciones entre colecciones:

Datos embebidos:

En Artículos, embebemos "atributos" como objeto flexible e "inventario" como arreglo de subdocumentos. Esto es eficiente porque las consultas de filtro (CU-001) requieren acceder a características y stock simultáneamente, evitando joins costosos en grandes volúmenes de datos. Embeber reduce la latencia en lecturas, ideal para e-commerce donde las búsquedas son frecuentes y deben responder en menos de 2 segundos.

En Carritos, embebemos "items" como arreglo. Permite actualizaciones atómicas (como \$push o \$pull) en una sola operación, crucial para picos de tráfico y para mantener consistencia sin transacciones multi-documento complejas.

Datos referenciados

- "sucursal_id" en inventario de Artículos referencia a Sucursales.
Las sucursales son entidades independientes y raras de cambiar; referenciar evita duplicación y normaliza datos.
Usamos \$lookup en agregaciones si necesitamos unir detalles de sucursales.
En el filtro avanzado (CU-001), la aplicación puede recibir una solicitud HTTP GET que contenga un parámetro "sucursal" con el valor de la sucursal_id para evitar el \$lookup en este caso de uso.
- "articulo_id" en items de Carritos referencia a Artículos.
Justificación: Los productos pueden actualizarse (por ejemplo el precio), pero almacenamos "precio_unitario" embebido en el carrito para dar consistencia en el momento de agregar, evitando inconsistencias si el precio cambia. Esto es eficiente porque las actualizaciones en carritos no dependen de joins en tiempo real.

Con estas elecciones, buscamos un equilibrio entre embeber algunos datos (que no crecen tanto) para obtener velocidad donde es imprescindible, y usar referencias donde es posible para evitar redundancia excesiva.

2.3 Consultas comunes:

2.3.1 Filtro avanzado de artículos (CU-001):

Descripción

Buscar artículos por atributos (color="Rojo", dimensiones.ancho > 50) y stock en sucursales específicas (stock >=1 en sucursal_id X).

Ejemplo de consulta (Aggregation Pipeline):

```
db.articulos.aggregate([
  { $match: {
    "atributos.color": "Rojo",
    "inventario": { $elemMatch: { "sucursal_id": ObjectId("..."), "stock": {
$gte: 1 } } } }
  },
  { $project: { nombre: 1, precio: 1, atributos: 1, imagenes: 1 } } ]]);
```

Explicación

\$match filtra eficientemente; \$elemMatch busca en arreglos sin unwind innecesario. Para rendimiento, indexar "atributos.color" y "inventario.sucursal_id".

2.3.2 Agregar/Actualizar ítem en carrito (CU-002):

Descripción

Operación atómica para agregar o incrementar cantidad, actualizando total y ultima_actividad.

Ejemplo de consulta:

```
db.carritos.updateOne(
  { _id: ObjectId("...") },
  { $push: { items: { articulo_id: ObjectId("..."), cantidad: 1,
precio_unitario: 10, subtotal: 10 } },
  $inc: { total: 10 },
  $set: { ultima_actividad: new Date() }
}
```

Explicación

Esta consulta guarda el precio unitario al momento de realizarse, para evitar que el valor del carrito se modifique con posibles modificaciones de precios de artículos. Asimismo, incrementa el valor de “total” por el subtotal agregado/actualizado. Y actualiza “ultima_actividad” para el control de TTL de 30 días de los carritos.